



Dominator

ALGORITHM PROJECT

NOUR IBRAHIM SALAMA
GANNA EMAD ABDULAZIZ
MOHAMED ALAA SALEH
SARA HAMDY AHMED
GIOVANY GEORGE
ABDULLAH NASSER

1.1- Non Recursive Algorithm

(Using Brute Force algorithm)

1. Pseudo code (Using Brute Force method):

function DOMINATOR(A, N):

IF $N == 0$:

 print "No dominator (empty array)"

 return

FOR i from 0 to $N - 1$:

 candidate $\leftarrow A[i]$

 count $\leftarrow 0$

 FOR j from 0 to $N - 1$:

 IF $A[j] == \text{candidate}$:

 count $\leftarrow \text{count} + 1$

 IF count $> N / 2$:

 print candidate, i

 return

print "No dominator found."

2. Analysis and complexity

→Time Complexity (by loop counting)

Pass 1: Outer Loop (Candidate Selection)

Number of iterations: N

Cost per iteration: $O(1)$

$$T_1(N) = N \cdot O(1) = O(N)$$

Pass 2: Inner Loop (Candidate Verification)

Number of iterations: N for each outer iteration

Cost per iteration: $O(1)$

$$T_2(N) = N \cdot O(1) = O(N)$$

Total Time Complexity

$$T(N) = T_1(N) + T_2(N)$$

$$T(N) = O(N) + O(N) = O(N^2)$$

$$T(N) = O(N^2)$$

The algorithm performs two nested loops over the array. Since the inner loop runs N times for each iteration of the outer loop, the overall time complexity is $O(N^2)$, making it less efficient than the Recursive approach for large inputs.

→Space Complexity

Mathematical Representation

$$S(N) = O(1)$$

Space Complexity Explanation

No extra data structures like arrays or hash tables are used.

The memory usage does not depend on N.

Only a few integer variables are maintained throughout execution (candidate, count, i, j).

1.2- Non Recursive Algorithm

(Using Boyer-Moore algorithm)

1. Pseudo code (boyer -moore method)

```
FUNCTION dominator(A, N)
  candidate ← A[0]
  count ← 1
  FOR i ← 1 TO N-1 DO
    IF count = 0 THEN
      candidate ← A[i]
      count ← 1
    ELSE IF A[i] = candidate THEN
      count ← count + 1
    ELSE
      count ← count - 1
    END IF
  END FOR
  freq ← 0
  FOR i ← 0 TO N-1 DO
    IF A[i] = candidate THEN
      freq ← freq + 1
    END IF
  END FOR
  IF freq ≤ N / 2 THEN
    RETURN -1
  END IF
  FOR i ← 0 TO N-1 DO
    IF A[i] = candidate THEN
      print candidate, i
    END IF
```

2. Analysis and complexity

→Time Complexity (by loop counting)

Pass 1: Candidate Selection

- Number of iterations: $N - 1$
- Cost per iteration: $O(1)$

$$T_1(N) = (N - 1) \cdot O(1) = O(N)$$

Pass 2: Candidate Verification

- Number of iterations: N
- Cost per iteration: $O(1)$

$$T_2(N) = N \cdot O(1) = O(N)$$

Pass 3: Index Retrieval

- Number of iterations: at most N
- Cost per iteration: $O(1)$

$$T_3(N) \leq N \cdot O(1) = O(N)$$

Total Time Complexity

$$\begin{aligned} T(N) &= T_1(N) + T_2(N) + T_3(N) \\ T(N) &= O(N) + O(N) + O(N) = O(3N) \\ T(N) &= O(N) \end{aligned}$$

The algorithm performs three linear passes over the array, each taking $O(N)$ time. Since constant factors are ignored in Big-O notation, the overall time complexity is $O(N)$, making the algorithm efficient and scalable for large inputs.

→Space Complexity

Mathematical Representation

$$S(N) = O(1)$$

Explanation

- No extra data structures (like arrays, lists, or hash tables) are used.
- The memory usage does **not depend on N** .
- Only a few integer variables are maintained throughout execution.

2- Recursive Algorithm

1.Pseudo code

Algorithm findCandidate(A, N, i, candidate, count) {

 IF $i = N$ then return;

 IF count = 0 then {

 candidate $\leftarrow A[i]$;

 count $\leftarrow 1$; }

 Else If $A[i] = \text{candidate}$ then {

 count $\leftarrow \text{count} + 1$; }

 Else {

 count $\leftarrow \text{count} - 1$; }

 findCandidate(A, N, $i + 1$, candidate, count);}

Algorithm countFreq(A, N, candidate, i){

 IF $i = N$ then return 0;

 return $(A[i] = \text{candidate} ? 1 : 0) + \text{countFreq}(A, N, \text{candidate}, i + 1)$;

Algorithm findIndex(A, N, candidate, i){

 IF $i = N$ then return -1;

 IF $A[i] = \text{candidate}$ then return i;

 return findIndex(A, N, candidate, $i + 1$);}

Algorithm dominator(A, N){

 IF $N = 0$ then return -1;

 candidate $\leftarrow A[0]$;

 count $\leftarrow 1$;

 findCandidate(A, N, 1, candidate, count);

 freq $\leftarrow \text{countFreq}(A, N, \text{candidate}, 0)$;

 If freq > $N / 2$ then

 return findIndex(A, N, candidate, 0)

 return -1;}

2. Analysis and complexity

→ Time Complexity (by loop counting)

Pass 1: findCandidate (Candidate Selection)

- Number of recursive calls: $N-1$

- Cost per call: $O(1)$

$$T_1(N) = (N-1) \cdot O(1) = O(N)$$

Pass 2: countFreq (Candidate Verification)

- Number of recursive calls: N

- Cost per call: $O(1)$

$$T_2(N) = N \cdot O(1) = O(N)$$

Pass 3: findIndex (Index Retrieval)

- Number of recursive calls: at most N

- Cost per call: $O(1)$

$$T_3(N) \leq N \cdot O(1) = O(N)$$

Total Time Complexity

$$T(N) = T_1(N) + T_2(N) + T_3(N)$$

$$T(N) = O(N) + O(N) + O(N) = O(3N)$$

$$T(N) = O(N)$$

The algorithm uses three recursive functions, each making $O(N)$ recursive calls. Since constant factors are ignored in Big-O notation, the overall time complexity is $O(N)$, making the algorithm efficient for large inputs.

→Time Complexity (by recurrence relation iteration method)

1-findCandidate:

$$T(0) = c_0$$

$$T(n) = T(n-1) + c$$

Expand:

$$T(n) = T(n-1) + c$$

$$= T(n-2) + 2c$$

$$= T(n-3) + 3c$$

$$= T(n-k) + k \cdot c$$

Base case: $n - k = 0 \rightarrow k = n$

Substitute:

$$T(n) = T(0) + n \cdot c$$

$$= c_0 + n \cdot c$$

$$= \Theta(N)$$

2-countFreq:

$$T(0) = c_0$$

$$T(n) = T(n-1) + c$$

Expand:

$$T(n) = T(n-1) + c$$

$$= T(n-2) + 2c$$

$$= T(n-3) + 3c$$

$$= T(n-k) + k \cdot c$$

Base case: $n - k = 0 \rightarrow k = n$

Substitute:

$$T(n) = T(0) + n \cdot c$$

$$= c_0 + n \cdot c$$

$$= \Theta(N)$$

3-findIndex:

$$T(0) = c_0$$

$$T(n) = T(n-1) + c$$

Expand:

$$T(n) = T(n-1) + c$$

$$= T(n-2) + 2c$$

$$= T(n-3) + 3c$$

$$= T(n-k) + k \cdot c$$

Base case: $n - k = 0 \rightarrow k = n$

Substitute:

$$T(n) = T(0) + n \cdot c$$

$$= c_0 + n \cdot c$$

$$= O(N) \quad \leftarrow O \text{ not } \Theta \text{ (can stop early)}$$

Total complexity:

$$T(N) = T_{\text{findCandidate}}(N) + T_{\text{countFreq}}(N) + T_{\text{findIndex}}(N)$$

$$= \Theta(N) + \Theta(N) + O(N)$$

$$= O(3N)$$

$$= O(N)$$

→Space Complexity

Mathematical Representation

$$S(N) = O(N)$$

Space Complexity Explanation

- Each recursive call adds a new frame to the Call Stack.
- The recursive depth may grow up to N in the worst case
- Therefore memory usage depends on N , giving $O(N)$ space.

3- Detailed Comparison for both the Algorithms

Feature	Non-Recursive Algorithm	Recursive Algorithm
Programming Technique	Iterative (Loops)	Recursive Functions
Main Idea	Uses loops to find and verify the dominator	Uses recursive calls to perform the same operations
Candidate Selection	Using a for loop	Using findCandidate() recursively
Frequency Counting	Iterative traversal	Recursive traversal
Index Retrieval	Iterative search	Recursive search
Time Complexity	$O(N)$	$O(N)$
Space Complexity	$O(1)$	$O(N)$
Memory Usage	Very low memory usage	Higher memory usage due to Call Stack
Performance	Faster in practice	Slightly slower
Scalability	Better for large arrays	May cause stack overflow
Ease of Understanding	Easier to trace and debug	Harder to follow

Code Structure	Simpler and shorter	More modular but more complex
Extra Memory	No extra memory required	Uses recursive stack frames
Practical Usage	Preferred in real applications	Mostly educational
Risk of Stack Overflow	No	Yes
Readability	Clear and straightforward	More abstract

Overall Conclusion

Both algorithms successfully solve the Dominator problem using the Boyer-Moore Voting idea and achieve linear time complexity $O(N)$.

The non-recursive algorithm is more efficient in terms of memory because it uses constant space $O(1)$. It is also faster and more suitable for large datasets.

The recursive algorithm provides a recursive interpretation of the same logic and helps demonstrate understanding of recursion concepts, but it consumes additional memory due to recursive call stack usage.

Therefore, the non-recursive solution is generally considered the better practical approach, while the recursive solution is useful for educational and conceptual purposes.